

ALTER TABLE

ALTER modifies an existing table.

```
ALTER TABLE Student ...
```

it generally performs the following:

1. Validates the SQL.
2. Checks user permissions.
3. Acquires metadata locks on the table.
4. Determines whether the requested change can be done instantly, in-place, or requires a table rebuild.
5. Updates the data dictionary.
6. Releases the locks.

ADD COLUMN

The most common ALTER operation.

Syntax

```
ALTER TABLE table_name  
ADD COLUMN column_name datatype;
```

Example:

```
ALTER TABLE Student  
ADD COLUMN Phone VARCHAR(15);
```

Add at a specific position

```
ALTER TABLE Student
ADD COLUMN Phone VARCHAR(15)
AFTER Name;
```

or

```
ALTER TABLE Student
ADD COLUMN RollNo INT
FIRST;
```

These positioning clauses are MySQL-specific.

Internal Working

If the new column allows NULL or has a compatible default, newer MySQL versions can often add it without rewriting every row.

Otherwise, MySQL may need to rebuild the table.

MODIFY COLUMN

Used to change the definition of a column.

Current	Salary INT
Need	Salary BIGINT
Use	ALTER TABLE Employee MODIFY COLUMN Salary BIGINT;

Another example

```
ALTER TABLE Student
MODIFY COLUMN Name VARCHAR(200);
```

You can also modify constraints

```
ALTER TABLE Student
MODIFY COLUMN Name VARCHAR(100) NOT NULL;
```

Internal Working

MySQL checks whether existing data is compatible with the new definition.

For example,

changing `VARCHAR(100)` to `VARCHAR(200)` is generally straightforward.

Changing `VARCHAR(100)` to `INT` may fail if existing values cannot be converted.

CHANGE COLUMN

This is frequently asked in interviews.

`CHANGE COLUMN` can

- rename the column
- change datatype
- change constraints

all at once.

```
ALTER TABLE table_name
CHANGE COLUMN
old_name
new_name
new_definition;
```

```
ALTER TABLE Employee
CHANGE COLUMN Salary MonthlySalary DECIMAL(10,2);
```

With `CHANGE COLUMN`, you must repeat the complete new column definition, even if only the name changes

MODIFY	CHANGE
Cannot rename	Can rename

MODIFY	CHANGE
Changes datatype/constraints	Can change name, datatype, constraints
Simpler	More flexible

RENAME COLUMN

Introduced in newer MySQL versions.

Syntax

```
ALTER TABLE Student
RENAME COLUMN Name TO FullName;
```

DROP COLUMN

Removes a column permanently.

Syntax

```
ALTER TABLE Student
DROP COLUMN Address;
```

ADD CONSTRAINT

Constraints enforce rules.

Example

Suppose Email should always be unique.

```
ALTER TABLE Student
ADD CONSTRAINT uq_student_email
UNIQUE (Email);
```

Another example

```
ALTER TABLE Employee  
ADD CONSTRAINT pk_employee  
PRIMARY KEY(EmployeeID);
```

DROP CONSTRAINT

Removing constraints depends on the constraint type.

Example (dropping a UNIQUE index created by a constraint)

```
ALTER TABLE Student  
DROP INDEX uq_student_email;
```

Dropping a foreign key:

```
ALTER TABLE Enrollment  
DROP FOREIGN KEY fk_student;
```

Dropping a primary key:

```
ALTER TABLE Employee  
DROP PRIMARY KEY;
```

Unlike SQL Server, MySQL doesn't have a single generic DROP CONSTRAINT statement for all constraint types.